Spark Practice

CONTENTS

1. Prerequisites 2
2. Task 2

PREREQUISITES

- Install Spark locally using one of the methods described here or in Docker.
- Create a Spark ETL job to read data from a local storage. You can find the data in the Spark Practice—Dataset file on the page with the task description.

TASK

- Check restaurant data for incorrect (null) values (latitude and longitude). For incorrect values, map latitude and longitude from the OpenCage Geocoding API in a job via the REST API.
- Generate a geohash by latitude and longitude using a geohash library like geohash-java. Your geohash should be four characters long and placed in an extra column.
- Left-join weather and restaurant data using the four-character geohash. Make sure to avoid data multiplication and keep your job idempotent.

Note: Development and testing should be done locally in your IDE environment.

- Store the enriched data (i.e., the joined data with all the fields from both datasets) in the local file system, preserving data partitioning in the parquet format.

You are expected to:

- Upload the source code and implement tests
- Upload your fully documented homework with screenshots and comments in the task Readme MD file with the repo link.

https://github.com/abdullagaukhar/spark-practice

[1]:
```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TestSpark") \
    .getOrCreate()

print("Spark version:", spark.version)
print("Всё работает! 🎉")
```

```
Spark version: 3.5.0
Всё работает! 🎉
```

[2]:
```
df = spark.read.csv(
    "/home/jovyan/work/restaurant_csv/",
    header=True,
    inferSchema=True
)

print("Количество строк:", df.count())
print("Колонки:", df.columns)
df.show(5)
```

```
Количество строк: 1997
Колонки: ['id', 'franchise_id', 'franchise_name', 'restaurant_franchise_id', 'country', 'city', 'lat', 'lng']
+------------+------------+----------------+----------------------+-------+----------+------+-------+
|          id|franchise_id|  franchise_name|restaurant_franchise_id|country|      city|   lat|    lng|
+------------+------------+----------------+----------------------+-------+----------+------+-------+
|197568495625|          10|The Golden Spoon|                 24784|     US|   Decatur|34.578|-87.021|
| 17179869242|          59|     Azalea Cafe|                 10902|     FR|     Paris|48.861|  2.368|
|214748364826|          27| The Corner Cafe|                 92040|     US|Rapid City| 44.08|-103.25|
|154618822706|          51|    The Pizzeria|                 41484|     AT|    Vienna|48.213| 16.413|
|163208757312|          65|   Chef's Corner|                 96638|     GB|    London|51.495| -0.191|
+------------+------------+----------------+----------------------+-------+----------+------+-------+
only showing top 5 rows
```

[3]:
```
from pyspark.sql.functions import col

nulls = df.filter(col("lat").isNull() | col("lng").isNull())
print("Строк с null координатами:", nulls.count())
nulls.show()
```

```
Строк с null координатами: 1
+-----------+------------+--------------+----------------------+-------+------+----+----+
|         id|franchise_id|franchise_name|restaurant_franchise_id|country|  city| lat| lng|
+-----------+------------+--------------+----------------------+-------+------+----+----+
|85899345920|           1|       Savoria|                 18952|     US|Dillon|NULL|NULL|
+-----------+------------+--------------+----------------------+-------+------+----+----+
```

[4]:
```python
import requests

API_KEY = "59707b01457a4ac3b1e2ee35922be98c"

def get_coordinates(city, country):
    url = f"https://api.opencagedata.com/geocode/v1/json"
    params = {
        "q": f"{city}, {country}",
        "key": API_KEY,
        "limit": 1
    }
    response = requests.get(url, params=params)
    data = response.json()

    if data["results"]:
        lat = data["results"][0]["geometry"]["lat"]
        lng = data["results"][0]["geometry"]["lng"]
        return lat, lng
    return None, None

# Проверяем для Dillon, US
lat, lng = get_coordinates("Dillon", "US")
print(f"Dillon, US -> lat: {lat}, lng: {lng}")
```

```
Dillon, US -> lat: 34.4014089, lng: -79.3864339
```

[5]:
```python
from pyspark.sql.functions import when, lit

df_fixed = df.withColumn(
    "lat",
    when(col("lat").isNull(), lit(34.4014089)).otherwise(col("lat"))
).withColumn(
    "lng",
    when(col("lng").isNull(), lit(-79.3864339)).otherwise(col("lng"))
)

# Проверяем что null больше нет
print("Null после исправления:", df_fixed.filter(col("lat").isNull() | col("lng").isNull()).count())

# Смотрим на исправленную строку
df_fixed.filter(df_fixed.id == 85899345920).show()
```

```
Null после исправления: 0
+-----------+------------+--------------+-----------------------+-------+------+----------+-----------+
|         id|franchise_id|franchise_name|restaurant_franchise_id|country|  city|       lat|        lng|
+-----------+------------+--------------+-----------------------+-------+------+----------+-----------+
|85899345920|           1|       Savoria|                  18952|     US|Dillon|34.4014089|-79.3864339|
+-----------+------------+--------------+-----------------------+-------+------+----------+-----------+
```

[6]:
```
import subprocess
subprocess.run(["pip", "install", "pygeohash"], capture_output=True)
print("Установлено!")
```

```
Установлено!
```

[7]:
```
import pygeohash as gh
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Функция которая кодирует координаты в geohash 4 символа
def encode_geohash(lat, lng):
    if lat is None or lng is None:
        return None
    return gh.encode(lat, lng, precision=4)

# Регистрируем функцию в Spark
geohash_udf = udf(encode_geohash, StringType())

# Добавляем колонку geohash
df_with_geohash = df_fixed.withColumn("geohash", geohash_udf(col("lat"), col("lng")))

df_with_geohash.select("franchise_name", "city", "lat", "lng", "geohash").show(5)
```

```
+----------------+----------+------+-------+-------+
|  franchise_name|      city|   lat|    lng|geohash|
+----------------+----------+------+-------+-------+
|The Golden Spoon|   Decatur|34.578|-87.021|   dn4h|
|     Azalea Cafe|     Paris|48.861|  2.368|   u09t|
| The Corner Cafe|Rapid City| 44.08|-103.25|   9xyd|
|    The Pizzeria|    Vienna|48.213| 16.413|   u2ed|
|   Chef's Corner|    London|51.495| -0.191|   gcpu|
+----------------+----------+------+-------+-------+
only showing top 5 rows
```

Would you like to receive official Jupyter new
Please read the privacy policy.

[9]:
```
weather_df = spark.read \
    .option("basePath", "/home/jovyan/work/weathers/weather_1/") \
    .parquet("/home/jovyan/work/weathers/weather_1/")

print("Количество строк:", weather_df.count())
print("Колонки:", weather_df.columns)
weather_df.show(5)
```

```
Количество строк: 4817180
Колонки: ['lng', 'lat', 'avg_tmpr_f', 'avg_tmpr_c', 'wthr_date', 'year', 'month', 'day']
+--------+-------+----------+----------+----------+----+-----+---+
|     lng|    lat|avg_tmpr_f|avg_tmpr_c| wthr_date|year|month|day|
+--------+-------+----------+----------+----------+----+-----+---+
|-111.202|18.7496|      81.0|      27.2|2016-10-04|2016|   10|  4|
|-111.155| 18.755|      81.0|      27.2|2016-10-04|2016|   10|  4|
|-111.107|18.7604|      80.9|      27.2|2016-10-04|2016|   10|  4|
|-111.059|18.7657|      80.9|      27.2|2016-10-04|2016|   10|  4|
|-111.012|18.7711|      80.9|      27.2|2016-10-04|2016|   10|  4|
+--------+-------+----------+----------+----------+----+-----+---+
only showing top 5 rows
```

[10]:
```python
from functools import reduce
from pyspark.sql import DataFrame

weather_dfs = []
for i in range(1, 9):
    path = f"/home/jovyan/work/weathers/weather_{i}/"
    df_temp = spark.read.option("basePath", path).parquet(path)
    weather_dfs.append(df_temp)

weather_all = reduce(DataFrame.union, weather_dfs)
print("Всего строк погоды:", weather_all.count())
print("Колонки:", weather_all.columns)
```

```
Всего строк погоды: 37333145
Колонки: ['lng', 'lat', 'avg_tmpr_f', 'avg_tmpr_c', 'wthr_date', 'year', 'month', 'day']
```

[11]:
```python
weather_with_geohash = weather_all.withColumn(
    "geohash",
    geohash_udf(col("lat"), col("lng"))
)

print("Погода с geohash:")
weather_with_geohash.select("lat", "lng", "avg_tmpr_c", "wthr_date", "geohash").show(5)
```

```
Погода с geohash:
+-------+--------+----------+----------+-------+
|    lat|     lng|avg_tmpr_c| wthr_date|geohash|
+-------+--------+----------+----------+-------+
|18.7496|-111.202|      27.2|2016-10-04|   9e2f|
| 18.755|-111.155|      27.2|2016-10-04|   9e2f|
|18.7604|-111.107|      27.2|2016-10-04|   9e2f|
|18.7657|-111.059|      27.2|2016-10-04|   9e34|
|18.7711|-111.012|      27.2|2016-10-04|   9e34|
+-------+--------+----------+----------+-------+
only showing top 5 rows
```

[3]:
```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit, udf, avg
from pyspark.sql.types import StringType
from functools import reduce
from pyspark.sql import DataFrame
import requests
import pygeohash as gh

# Запуск Spark с памятью
spark = SparkSession.builder \
    .appName("SparkPractice") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()

# Читаем рестораны
df = spark.read.csv("/home/jovyan/work/restaurant_csv/", header=True, inferSchema=True)

# Исправляем null координаты
lat, lng = 34.4014089, -79.3864339

df_fixed = df.withColumn("lat", when(col("lat").isNull(), lit(lat)).otherwise(col("lat"))) \
            .withColumn("lng", when(col("lng").isNull(), lit(lng)).otherwise(col("lng")))

# Добавляем geohash
def encode_geohash(lat, lng):
    if lat is None or lng is None:
        return None
    return gh.encode(lat, lng, precision=4)

geohash_udf = udf(encode_geohash, StringType())
df_with_geohash = df_fixed.withColumn("geohash", geohash_udf(col("lat"), col("lng")))

print("Рестораны готовы:", df_with_geohash.count())
df_with_geohash.show(3)
```

```
Рестораны готовы: 1997
+------------+------------+----------------+-----------------------+-------+----------+------+-------+-------+
|          id|franchise_id|  franchise_name|restaurant_franchise_id|country|      city|   lat|    lng|geohash|
+------------+------------+----------------+-----------------------+-------+----------+------+-------+-------+
|197568495625|          10|The Golden Spoon|                  24784|     US|   Decatur|34.578|-87.021|   dn4h|
| 17179869242|          59|     Azalea Cafe|                  10902|     FR|     Paris|48.861|  2.368|   u09t|
|214748364826|          27| The Corner Cafe|                  92040|     US|Rapid City| 44.08|-103.25|   9xyd|
+------------+------------+----------------+-----------------------+-------+----------+------+-------+-------+
only showing top 3 rows
```

[4]:
```python
# Читаем погоду
weather_dfs = []
for i in range(1, 9):
    path = f"/home/jovyan/work/weathers/weather_{i}/"
    df_temp = spark.read.option("basePath", path).parquet(path)
    weather_dfs.append(df_temp)

weather_all = reduce(DataFrame.union, weather_dfs)
print("Погода загружена, строк:", weather_all.count())
```

```
Погода загружена, строк: 37333145
```

[1]:
```python
import subprocess
subprocess.run(["pip", "install", "pygeohash"], capture_output=True)

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit, udf, avg
from pyspark.sql.types import StringType
import pygeohash as gh

spark = SparkSession.builder \
    .appName("SparkPractice") \
    .config("spark.driver.memory", "6g") \
    .config("spark.sql.shuffle.partitions", "20") \
    .getOrCreate()

def encode_geohash(lat, lng):
    if lat is None or lng is None:
        return None
    return gh.encode(lat, lng, precision=4)

geohash_udf = udf(encode_geohash, StringType())

# Рестораны
df = spark.read.csv("/home/jovyan/work/restaurant_csv/", header=True, inferSchema=True)
df_fixed = df.withColumn("lat", when(col("lat").isNull(), lit(34.4014089)).otherwise(col("lat"))) \
             .withColumn("lng", when(col("lng").isNull(), lit(-79.3864339)).otherwise(col("lng")))
df_with_geohash = df_fixed.withColumn("geohash", geohash_udf(col("lat"), col("lng")))
df_with_geohash.cache()

restaurant_geohashes = [row.geohash for row in df_with_geohash.select("geohash").distinct().collect()]
print(f"Spark готов! Geohash ресторанов: {len(restaurant_geohashes)}")
```

```python
# Обрабатываем по одной папке
for i in range(1, 9):
    print(f"Обрабатываем weather_{i}...")
    path = f"/home/jovyan/work/weathers/weather_{i}/"

    df_temp = spark.read.option("basePath", path).parquet(path)
    df_temp = df_temp.withColumn("geohash", geohash_udf(col("lat"), col("lng")))
    df_temp = df_temp.filter(col("geohash").isin(restaurant_geohashes))

    df_agg = df_temp.groupBy("geohash", "wthr_date") \
        .agg(avg("avg_tmpr_c").alias("avg_tmpr_c"),
             avg("avg_tmpr_f").alias("avg_tmpr_f"))

    result = df_with_geohash.join(df_agg, on="geohash", how="left")

    result.write \
        .mode("append") \
        .partitionBy("country") \
        .parquet("/home/jovyan/work/output/")

    spark.catalog.clearCache()
    df_with_geohash.cache()
    print(f"weather_{i} сохранён!")

print("Всё готово! 🎉")
```

```
Spark готов! Geohash ресторанов: 692
Обрабатываем weather_1...
weather_1 сохранён!
Обрабатываем weather_2...
weather_2 сохранён!
Обрабатываем weather_3...
weather_3 сохранён!
Обрабатываем weather_4...
weather_4 сохранён!
Обрабатываем weather_5...
weather_5 сохранён!
Обрабатываем weather_6...
weather_6 сохранён!
Обрабатываем weather_7...
weather_7 сохранён!
Обрабатываем weather_8...
weather_8 сохранён!
Всё готово! 🎉
```

[2]:
```python
result_check = spark.read.parquet("/home/jovyan/work/output/")
print("Строк в результате:", result_check.count())
print("Колонки:", result_check.columns)
result_check.show(5)
```

```
Строк в результате: 66697
Колонки: ['geohash', 'id', 'franchise_id', 'franchise_name', 'restaurant_franchise_id', 'city', 'lat', 'lng', 'wthr_date', 'avg_tmpr_c', 'avg_tmpr_f',
'country']
+-------+------------+------------+-----------------+-----------------------+--------+------+--------+----------+------------------+-----------------
+-------+
|geohash|          id|franchise_id|    franchise_name|restaurant_franchise_id|    city|   lat|     lng| wthr_date|        avg_tmpr_c|       avg_tmpr_f
|country|
+-------+------------+------------+-----------------+-----------------------+--------+------+--------+----------+------------------+-----------------
+-------+
|   9t9p|111669149700|           5|  The Hungry Hippo|                  84488|  Tucson| 32.22|-110.877|2016-10-08|            25.632|78.13600000000001
|     US|
|   9t9p|111669149700|           5|  The Hungry Hippo|                  84488|  Tucson| 32.22|-110.877|2016-10-07|            24.868|76.77599999999998
|     US|
|   9t9p|111669149700|           5|  The Hungry Hippo|                  84488|  Tucson| 32.22|-110.877|2016-10-06|            22.156|            71.872
|     US|
|   9t9p|111669149700|           5|  The Hungry Hippo|                  84488|  Tucson| 32.22|-110.877|2016-10-05|22.028000000000002|71.65999999999998
|     US|
|   9q9x| 42949672973|          14|The Gourmet Garden|                  89646|Stockton|37.956|-121.328|2016-10-07|18.883333333333333|65.99166666666667
|     US|
+-------+------------+------------+-----------------+-----------------------+--------+------+--------+----------+------------------+-----------------
+-------+
only showing top 5 rows
```

[4]:
```python
# ===== ТЕСТЫ исправленные =====
print("Запускаем тесты...\n")

# Тест 1 — нет null координат
nulls_count = df_with_geohash.filter(col("lat").isNull() | col("lng").isNull()).count()
assert nulls_count == 0
print("✅ Тест 1 пройден: null координат нет")

# Тест 2 — geohash ровно 4 символа
from pyspark.sql.functions import length
wrong_geohash = df_with_geohash.filter(length(col("geohash")) != 4).count()
assert wrong_geohash == 0
print("✅ Тест 2 пройден: все geohash длиной 4 символа")

# Тест 3 — результат не пустой
result_count = result_check.count()
assert result_count > 0
print(f"✅ Тест 3 пройден: в результате {result_count} строк")

# Тест 4 — количество ресторанов не изменилось
restaurant_count = df_with_geohash.select("id").distinct().count()
result_restaurant_count = result_check.select("id").distinct().count()
assert restaurant_count == result_restaurant_count
print(f"✅ Тест 4 пройден: все {restaurant_count} ресторанов в результате")

# Тест 5 — файл сохранён в parquet
import os
assert os.path.exists("/home/jovyan/work/output/")
print("✅ Тест 5 пройден: output папка существует")

print("\n🎉 Все тесты пройдены!")
```

```
Запускаем тесты...

✅ Тест 1 пройден: null координат нет
✅ Тест 2 пройден: все geohash длиной 4 символа
✅ Тест 3 пройден: в результате 66697 строк
✅ Тест 4 пройден: все 1997 ресторанов в результате
✅ Тест 5 пройден: output папка существует
```